

Exp. No. 1

Develop a lexical Analyzer to identify identifiers, constants, operators using C program.

```
#include <stdio.h>

#include <ctype.h>

int main()
{
    int i, ic = 0, cc = 0, oc = 0, num;
    char str[100];
    char identifiers[100];
    int constants[100];
    char operators[100];
    printf("enter the string : ");
    fgets(str, sizeof(str), stdin);

    for(i = 0; str[i] != '\0'; i++)
    {
        if(isalpha(str[i]))
        {
            identifiers[ic++] = str[i];
        }
        else if(isdigit(str[i]))
        {
            num = 0;
            while(isdigit(str[i]))
            {
                num = num * 10 + (str[i] - '0');
```

```

        i++;
    }
    i--;
    constants[cc++] = num;
}
else if(str[i] == '+' || str[i] == '-' ||
        str[i] == '*' || str[i] == '/' ||
        str[i] == '=')
{
    operators[oc++] = str[i];
}
}
printf("identifiers : ");
for(i = 0; i < ic; i++)
    printf("%c ", identifiers[i]);
printf("\nconstants : ");
for(i = 0; i < cc; i++)
    printf("%d ", constants[i]);
printf("\noperators : ");
for(i = 0; i < oc; i++)
    printf("%c ", operators[i]);

return 0;
}

```

```

main.c
28     constants[cc++] = num;
29     }
30     else if(str[i] == '+' || str[i] == '-' ||
31             str[i] == '*' || str[i] == '/' ||
32             str[i] == '=')
33     {
34         operators[oc++] = str[i];
35     }
36 }
37 printf("identifiers : ");
38 for(i = 0; i < ic; i++)
39     printf("%c ", identifiers[i]);
40 printf("\nconstants : ");
41 for(i = 0; i < cc; i++)
42     printf("%d ", constants[i]);
43 printf("\noperators : ");
44 for(i = 0; i < oc; i++)
45     printf("%c ", operators[i]);
46
47 return 0;
48 }

```

Output

```

enter the string : a=b+c*d
identifiers : a b c d
constants :
operators : = + *

=== Code Execution Successful ===

```

Exp. No. 2

Develop a lexical Analyzer to identify whether a given line is a comment or not using C

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char com[100];
```

```
    int i, a = 0;
```

```
    printf("Enter comment: ");
```

```
    fgets(com, sizeof(com), stdin);
```

```
    if (com[0] == '/')
```

```
    {
```

```
        if (com[1] == '/')
```

```
        {
```

```
            printf("It is a comment");
```

```
        }
```

```
    else if (com[1] == '*')
```

```
    {
```

```
for (i = 2; com[i] != '\0'; i++)
{
    if (com[i] == '*' && com[i + 1] == '/')
    {
        printf("It is a comment");
        a = 1;
        break;
    }
}
if (a == 0)
{
    printf("It is not a comment");
}
}
else
{
    printf("It is not a comment");
}
}
else
{
    printf("It is not a comment");
}
}
return 0;
}
```

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     char com[100];
6     int i, a = 0;
7
8     printf("Enter comment: ");
9     fgets(com, sizeof(com), stdin);
10
11     if (com[0] == '/')
12     {
13         if (com[1] == '/')
14         {
15             printf("It is a comment");
16         }
17         else if (com[1] == '*')
18         {
19             for (i = 2; com[i] != '\0'; i++)
20             {
21                 if (com[i] == '*' && com[i + 1] == '/')
22                 {
23                     printf("It is a comment");
24                     break;
25                 }
26             }
27         }
28     }
29 }
```

Output

```
Enter comment: hello
It is not a comment

=== Code Execution Successful ===
```

Exp. No. 3

Design a lexical Analyzer for given language should ignore the redundant spaces, tabs and new lines and ignore comments using C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
int isKeyword(char buffer[])
```

```
{
```

```
    char keywords[][10] = {
```

```
        "main", "auto", "break", "case", "char", "const",
```

```
        "continue", "default", "do", "double", "else", "enum",
```

```
        "extern", "float", "for", "goto", "if", "int", "long",
```

```
        "register", "return", "short", "signed", "sizeof",
```

```
        "static", "struct", "switch", "typedef", "unsigned",
```

```
        "void", "printf", "while"
```

```
    };
```

```

int i;
for(i = 0; i < 32; i++)
{
    if(strcmp(keywords[i], buffer) == 0)
        return 1;
}
return 0;
}
int main()
{
    char str[500], buffer[50];
    char operators[] = "+-*/%=";
    int i, j = 0, k
    printf("Enter the program:\n");
    fgets(str, sizeof(str), stdin);
    for(i = 0; str[i] != '\0'; i++)
    {
        /* Check operators */
        for(k = 0; k < 6; k++)
        {
            if(str[i] == operators[k])
            {
                printf("%c is operator\n", str[i]); } }
        /* Build identifiers / keywords */
        if(isalnum(str[i]))
        {

```

```

        buffer[j++] = str[i];
    }
    else if(j != 0)
    {
        buffer[j] = '\0';

        j = 0;

        if(isKeyword(buffer))

            printf("%s is keyword\n", buffer);

        else

            printf("%s is identifier\n", buffer);

    }

return 0;
}

```

main.c

Share

Run

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int isKeyword(char buffer[])
6  {
7      char keywords[][10] = {
8          "main", "auto", "break", "case", "char", "const",
9          "continue", "default", "do", "double", "else", "enum",
10         "extern", "float", "for", "goto", "if", "int", "long",
11         "register", "return", "short", "signed", "sizeof",
12         "static", "struct", "switch", "typedef", "unsigned",
13         "void", "printf", "while"
14     };
15
16     int i;
17     for(i = 0; i < 32; i++)
18     {
19         if(strcmp(keywords[i], buffer) == 0)
20             return 1;
21     }
22     return 0;

```

Output

Enter the program:
flex_input.txt
main ()
{
int a, b, c ;
c = b + c;
printf ("%d" ,c) ;
}
flex is identifier
input is identifier
txt is identifier

=== Code Execution Successful ===

Exp. No. 4

Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular arithmetic operators using C

```
#include <stdio.h>

int main()
{
    char s[10];
    printf("Enter any operator: ");
    fgets(s, sizeof(s), stdin)
    switch(s[0])
    {
        case '>':
            if(s[1] == '=')
                printf("Greater than or equal");
            else
                printf("Greater than");
            break;
        case '<':
            if(s[1] == '=')
                printf("Less than or equal");
            else
                printf("Less than");
            break;
        case '=':
            if(s[1] == '=')
                printf("Equal to");
            else
```



```
        printf("Assignment");
    break
case '!':
    if(s[1] == '=')
        printf("Not Equal");
    else
        printf("Bit Not");
    break;
case '&':
    if(s[1] == '&')
        printf("Logical AND");
    else
        printf("Bitwise AND");
    break;
case '|':
    if(s[1] == '|')
        printf("Logical OR");
    else
        printf("Bitwise OR");
    break;
case '+':
    printf("Addition");
    break;
case '-':
    printf("Subtraction");
    break
```

```

    case '*':
        printf("Multiplication");
        break
    case '/':
        printf("Division");
        break;
    case '%':
        printf("Modulus");
        break;
    default:
        printf("Not an operator");
}
return 0;
}

```

main.c

Share

Run

Output

```

1  #include <stdio.h>
2
3  int main()
4  {
5      char s[10];
6
7      printf("Enter any operator: ");
8      fgets(s, sizeof(s), stdin);
9
10     switch(s[0])
11     {
12         case '>':
13             if(s[1] == '=')
14                 printf("Greater than or equal");
15             else
16                 printf("Greater than");
17             break;
18
19         case '<':
20             if(s[1] == '=')
21                 printf("Less than or equal");
22             else

```

Enter any operator: >=
Greater than or equal

=== Code Execution Successful ===

Exp. No. 5

Design a lexical Analyzer to find the number of whitespaces and newline characters using C.

```
#include <stdio.h>

int main()
{
    char str[100];

    int words = 0, lines = 0, characters = 0;

    int inWord = 0

    printf("Enter text (use ~ to end):\n");
    scanf("%99[^\n]", str);
    for (int i = 0; str[i] != '\0'; i++)
    {
        characters++;

        /* Count lines */
        if (str[i] == '\n')
        {
            lines++;
        }

        /* Count words */
        if (str[i] != ' ' && str[i] != '\t' && str[i] != '\n')
        {
            if (!inWord)
            {
                words++;
                inWord = 1;
            }
        }
    }
}
```

```

    }
    else
    {
        inWord = 0;
    }
}

/* If there is any text, number of lines = newlines + 1 */
if (characters > 0)
{
    lines++;
}

printf("Total number of words: %d\n", words);
printf("Total number of lines: %d\n", lines);
printf("Total number of characters: %d\n", characters)

return 0;
}

```

main.c	Run	Output
<pre> 1 #include <stdio.h> 2 3 int main() 4 { 5 char str[100]; 6 int words = 0, lines = 0, characters = 0; 7 int inWord = 0; 8 9 printf("Enter text (use ~ to end):\n"); 10 scanf("%99[^\n]", str); 11 12 for (int i = 0; str[i] != '\0'; i++) 13 { 14 characters++; 15 16 /* Count lines */ 17 if (str[i] == '\n') 18 { 19 lines++; 20 } 21 </pre>	Run	<pre> Enter text (use ~ to end): Hello world This is C language~ Total number of words: 6 Total number of lines: 2 Total number of characters: 30 === Code Execution Successful === </pre>